

Machine Learning for Computer Networking

Based on Slides from Nick Feamster, University of Chicago

Data Science = Modeling (“ML”) +
Preparation +
Deployment

Modeling:

Types of Machine Learning Problems

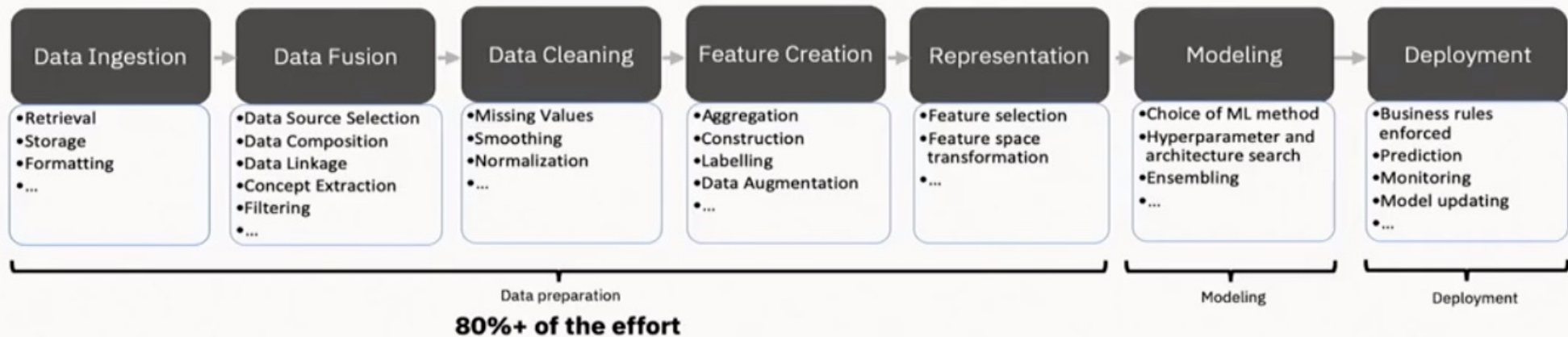
- **Supervised Learning**

- Regression: What is the value of something I'm trying to predict?
- Classification: What class/type does this data point belong to?
- Prediction: What is going to happen? What will happen if...?

- **Unsupervised Learning**

- Clustering: What are natural groups for this data?
- Dimensionality Reduction: What are the most important aspects of the data?
- Recommendation: "You might also like..."

Steps in the ML/Data Science Pipeline



Data preparation:

- **Affects the final predictive accuracy** -- generally *more* than the modeling step does!
- Like modeling, also *contains parameters* which **should be tuned**
- **80%+ of the effort** in a data project is in data preparation (some say 90%+)
- Not treated in textbooks: left as black art → gives rise to **many conceptual errors** in practice -- *most* errors in data science happen in data preparation

Credit: Alex Gray

Data Ingestion/Fusion/Exploration

- **Outliers (zero values, etc.):** May indicate errors
- **Distributions:** What stands out about the data?
- **Separators:** Are there features that naturally separate classes you are interested in?
- **Correlations:** Are certain features correlated with one another?
- **Temporal patterns:** Hourly? Daily? Weekly? Seasonal?
 - Very important for understanding whether a model trained today will be valid tomorrow!

Data Cleaning

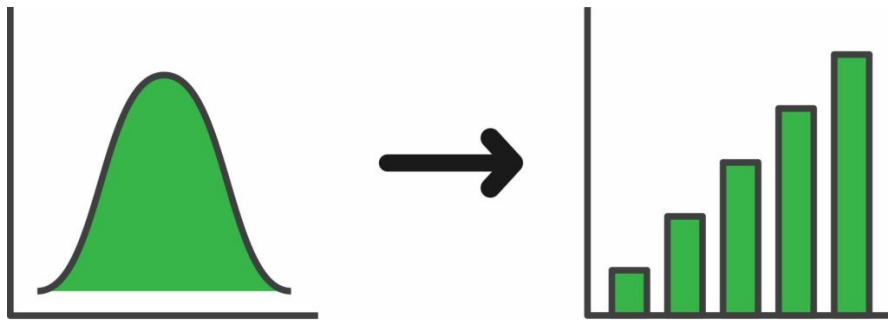
- Sanity checks
 - Missing values
 - Zero values
 - Negative values
 - Data of the wrong type
- Smoothing
- Normalization

Feature Engineering: Creation and Representation

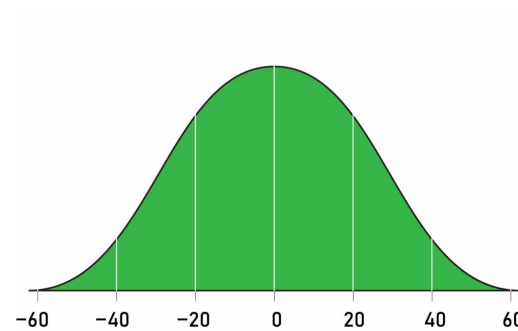
- Discretization
 - Map continuous-valued feature into "buckets"
- Sampling
 - Random, periodic, ...
- Transformations
 - Fourier transform
 - Encodings (e.g., one-hot encoding)
- Aggregation
 - Temporal: "Binning" quantitative values into discrete time bins
 - Spatial

Feature Engineering: Creation and Representation

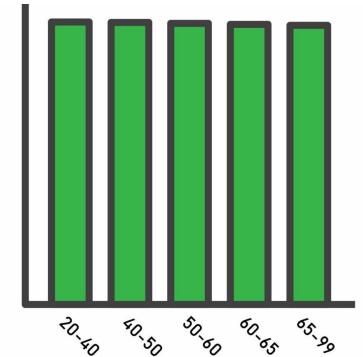
- Discretization
 - Map continuous-valued feature into "buckets"



Discretization Process



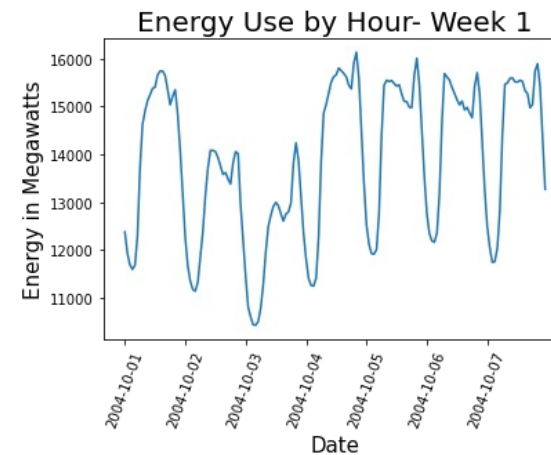
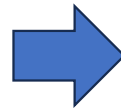
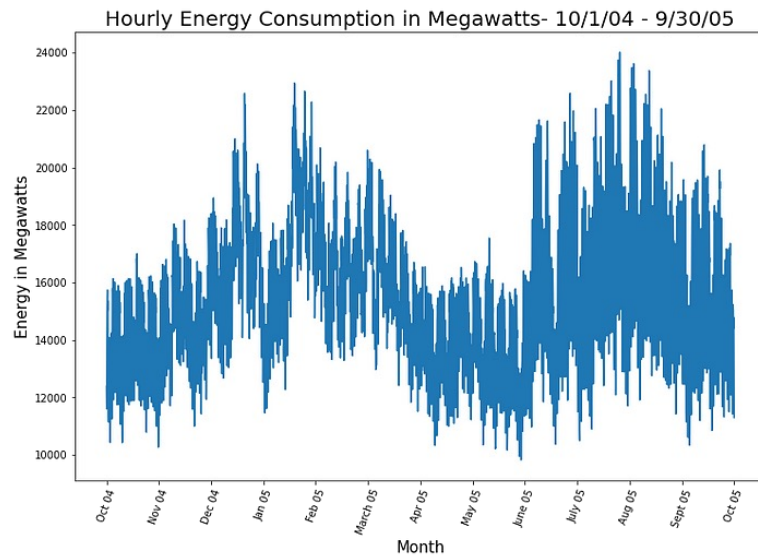
Equal width Discretization



Equal-Frequency Discretization

Feature Engineering: Creation and Representation

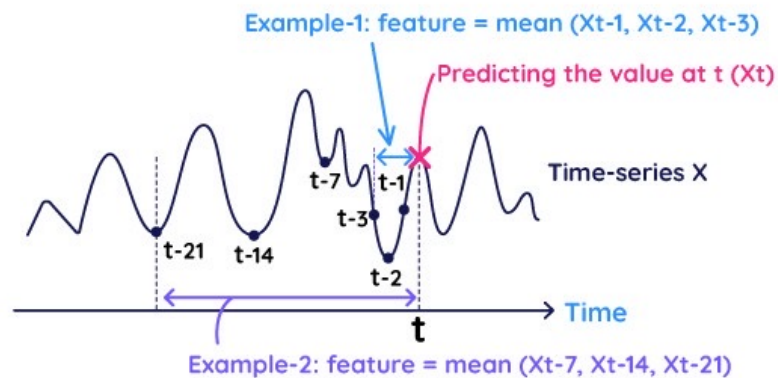
- Transformations
 - Fourier transform
 - Encodings (e.g., one-hot encoding)



Src: <https://nathaniel-lanier.medium.com/fourier-transforms-for-detecting-seasonality-and-feature-engineering-5a91fbfe6e7c>

Feature Engineering: Creation and Representation

- Aggregation
 - Temporal: "Binning" quantitative values into discrete time bins
 - Spatial



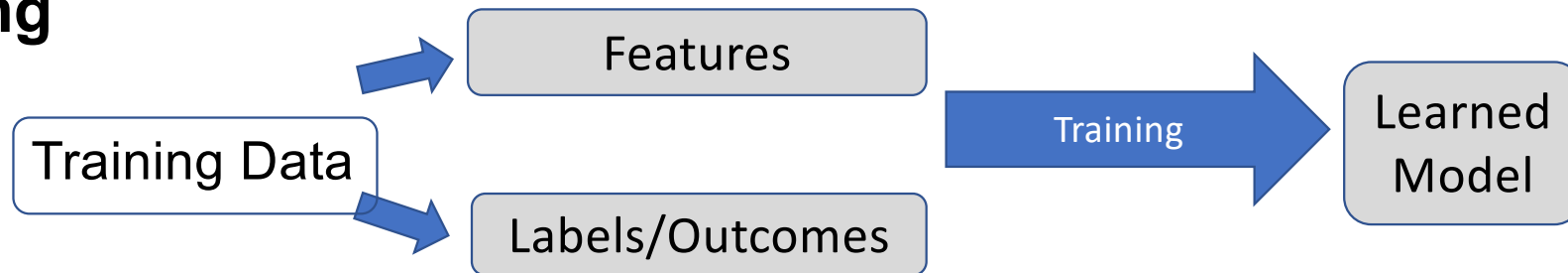
- Temporal Binning:
 - **Purpose:** Transforms continuous time values into discrete categories or bins, enabling analysis of patterns over time.
 - **Examples:** Grouping time data into hourly, daily, weekly, or monthly intervals.
 - **Benefits:** Captures temporal dependencies, simplifies time-series data, and can improve model performance.

Src: [link](#)

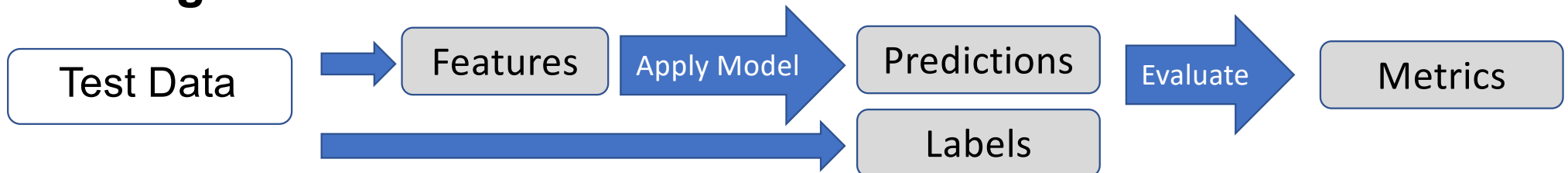
Modeling (Supervised Learning)

- Model Selection and Training
 - **Considerations:** Accuracy, complexity, training cost, assumptions about underlying data, ...
- Parameter Tuning

Training



Testing



Evaluation and Testing

- **Common:** Using Historical Data
 - Training and testing: Cross-validation
 - Accuracy metrics:
 - Accuracy, F1 Score, Receiver Operating Characteristic (FP/FN), Precision/Recall
 - Challenge: Doing better than just relearning the labeled dataset
- **Also:** Deployment/Field Approach
 - Harder!
 - Need deployment
 - Streaming model, real-time constraints, ...

Anaconda Installation

[Products](#)[Why Anaconda?](#)[Solutions](#)[Resources](#)[Company](#)[Contact Us](#)[Download](#)

Anaconda Individual Edition

The World's Most Popular Python/R Data Science Platform

[Download](#)

Anaconda 2020.02

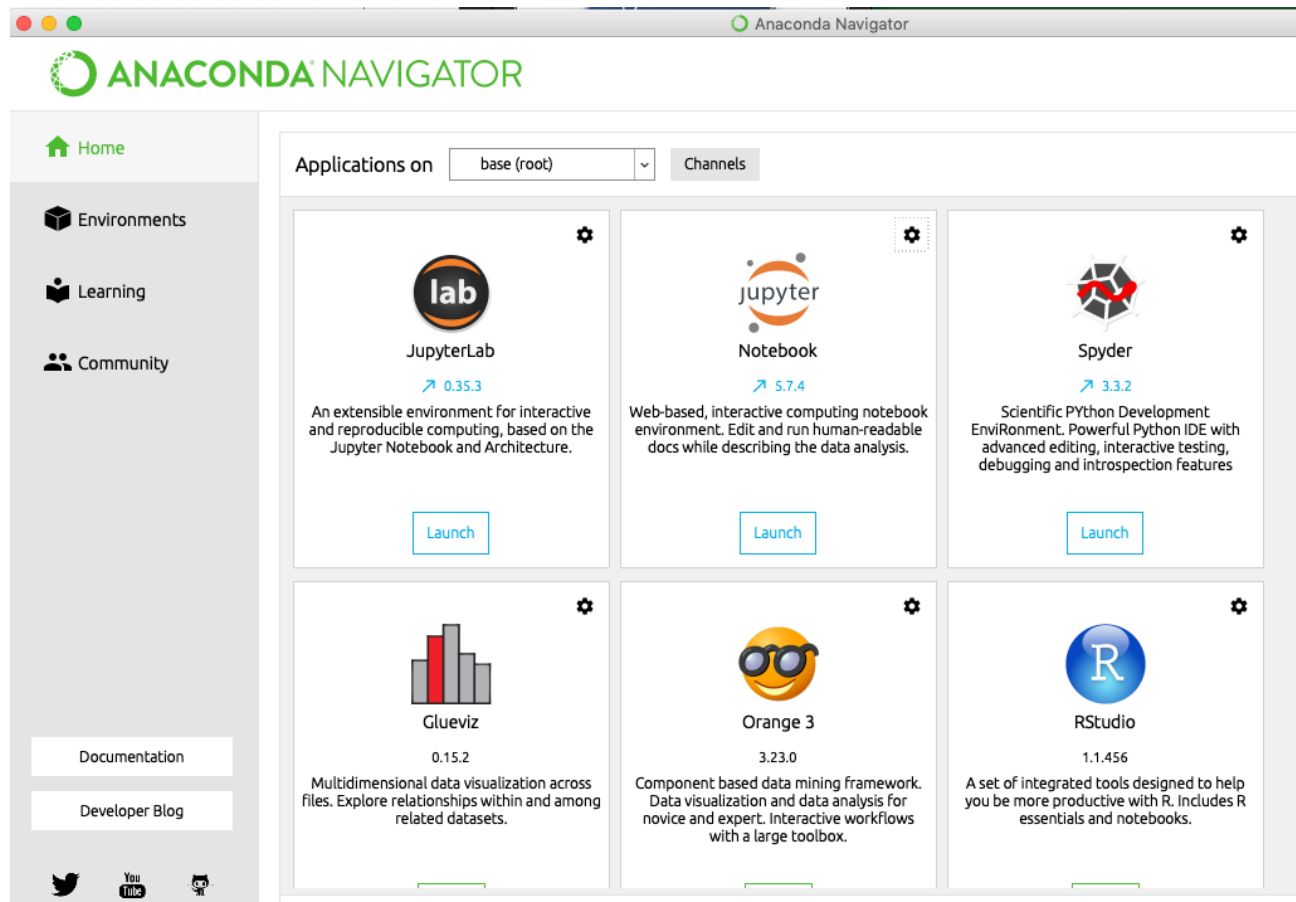
Python 3.7 version

[Download](#)

64-Bit Graphical Installer (442 MB)

64-Bit Command Line Installer (430 MB)

Jupyter Notebooks



Data Ingestion/Exploration

Python Data Libraries

- **NumPy**: multi-dimensional arrays, matrices, mathematical operations
- **SciPy**: linear algebra, differential equations, optimization, statistics
- **Pandas**: data structures and tools for table-line data
(reshaping, merging, sorting, slicing, etc.)
- **SciKit-Learn**: machine learning algorithms

Visualization and Plotting Libraries

- **Matplotlib**: 2D plotting library, similar to matlab-style plotting
- **Seaborn**: based on matplotlib, higher-level interface

NumPy Examples

- Creating vectors and matrices
- Describing a matrix
- Calculating average, variance, and standard deviation

Pandas Examples

- Loading data
 - From CSV
 - From JSON
- Wrangling Data
 - Describing the data
 - Selecting based on conditionals
 - Grouping rows by values

CTA - Ridership - 'L' Station Entries - Daily Totals

[View Data](#)[Visualize](#) ▾[Export](#)[API](#)

...

This list shows daily totals of ridership, by station entry, for each 'L' station dating back to 2001. Dataset shows entries at all turnstiles, combined, for each station. Daytypes are as follows: W=Weekday, A=Saturday, U=Sunday/Holiday. See attached readme file for information on how these numbers are calculated.

Updated
February 10, 2020

Data Provided by
Chicago Transit Authority

About this Dataset

Updated

February 10, 2020

Data Last Updated

February 10, 2020

Metadata Last Updated

February 10, 2020

Date Created

August 5, 2011

Metadata

Data Owner

Chicago Transit Authority

Attachments

 [ridershipreadme.txt](#)

Topics

Category

Transportation

Tags

[cta](#), [public transit](#), [ridership](#), [chicago transit authority](#)

Licensing and Attribution

License

The license for this dataset is unspecified

Source Link

<http://www.transitchicago.com> 

[Contact Dataset Owner](#)

What's in this Dataset?

Rows

989K

Columns

5

Model Selection and Cross Validation

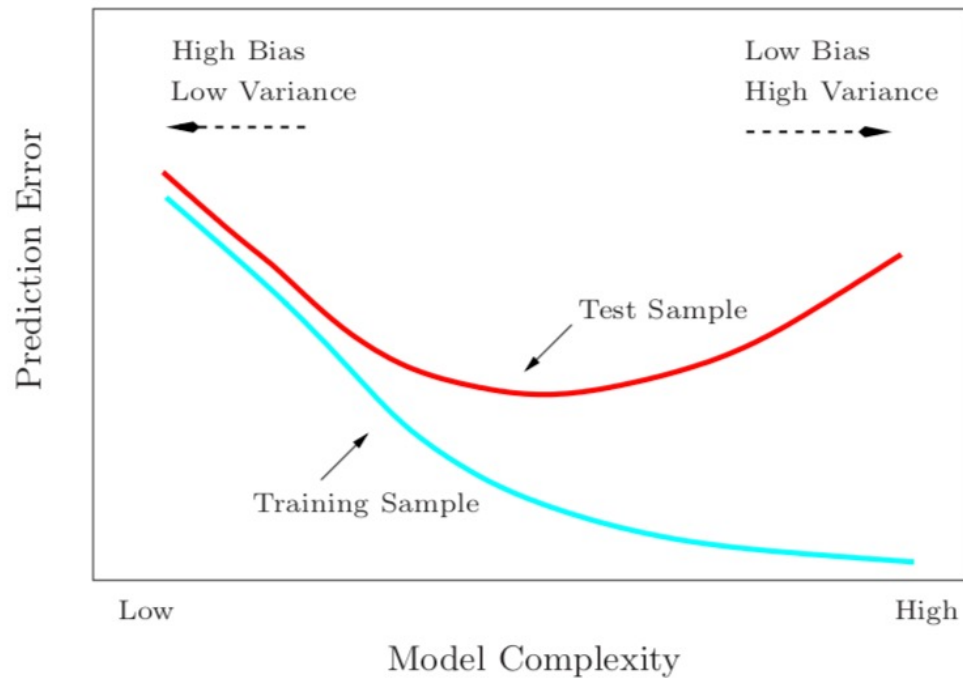
Model Selection

- Biggest question: **Does the machine learning algorithm work?**
- Need to evaluate the quality of models created through algorithms and training.
- Key to all evaluation approaches: **Never touch the test data!**
 - In practice, this can be a lot harder than it sounds

Problem

- **Getting the right balance between underfitting and overfitting.**
- Huge design space for the possibilities of all machine learning models.
- Lots of tunable “knobs” (called **hyperparameters**)
 - The lambda parameter in regularization of linear regression models
 - Choice of penalty functions, basis functions, ...
- The hyperparameters can be tweaked, but it can be hard to figure out what the right setting is because training error isn't informative.

Training Aims to Minimize Error



- If our training includes hyper-parameters, then optimization will include over-fitting.
- **Example:** High-degree polynomial expansion.

General Approach: Validation “Hold Out” Set

1. Prepare data: Split data into three sets

- **Training data:** learning/model training
- **Validation data:** For training hyper-parameters
- **Testing data:** only for testing

2. Optimize model on training data

- Using training data, optimize model to minimize bias on training set

3. Find optimal hyper-parameters

(meta-learning, hyper-parameter optimization)

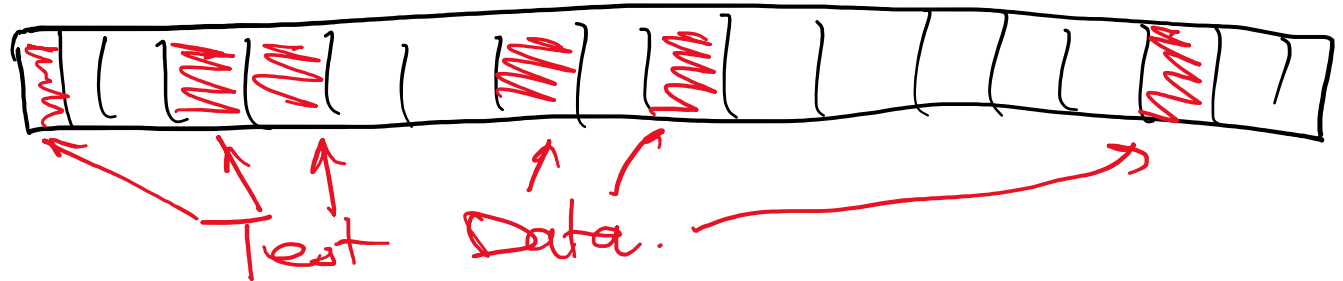
- Use validation data to tune parameters (polynomial degree, loss weight) to minimize error (e.g., MSE) on each validation sample.

How Large Should the Test Set Be?

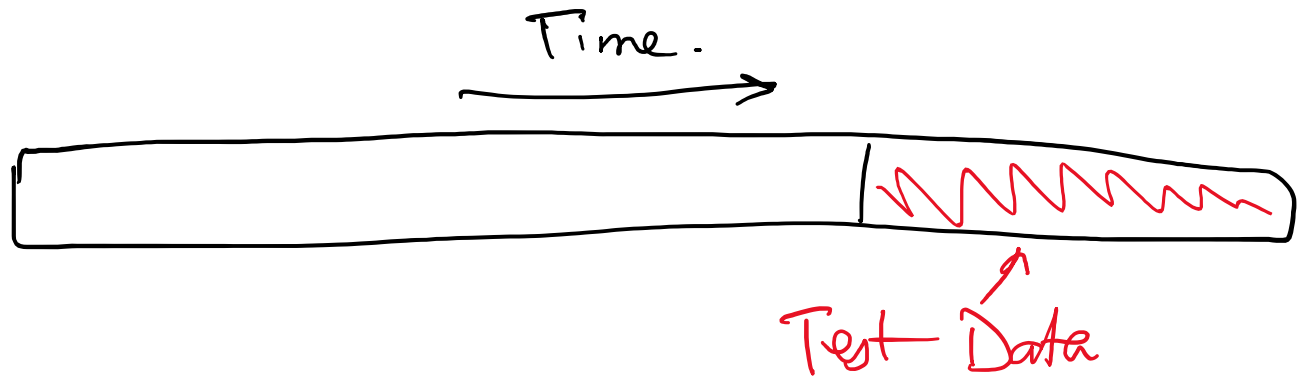
- No fixed rule, but there are some **tradeoffs**
 - **Test data** needs to be **large enough to estimate out-of-sample error**
 - **Training data** needs to be **large enough to train the model**
- **Rough Guideline:** Error estimation is inversely proportional to the square root of the size of the test set.
- **Common Approach:** 80% Training, 20% Testing

How to Split the Data?

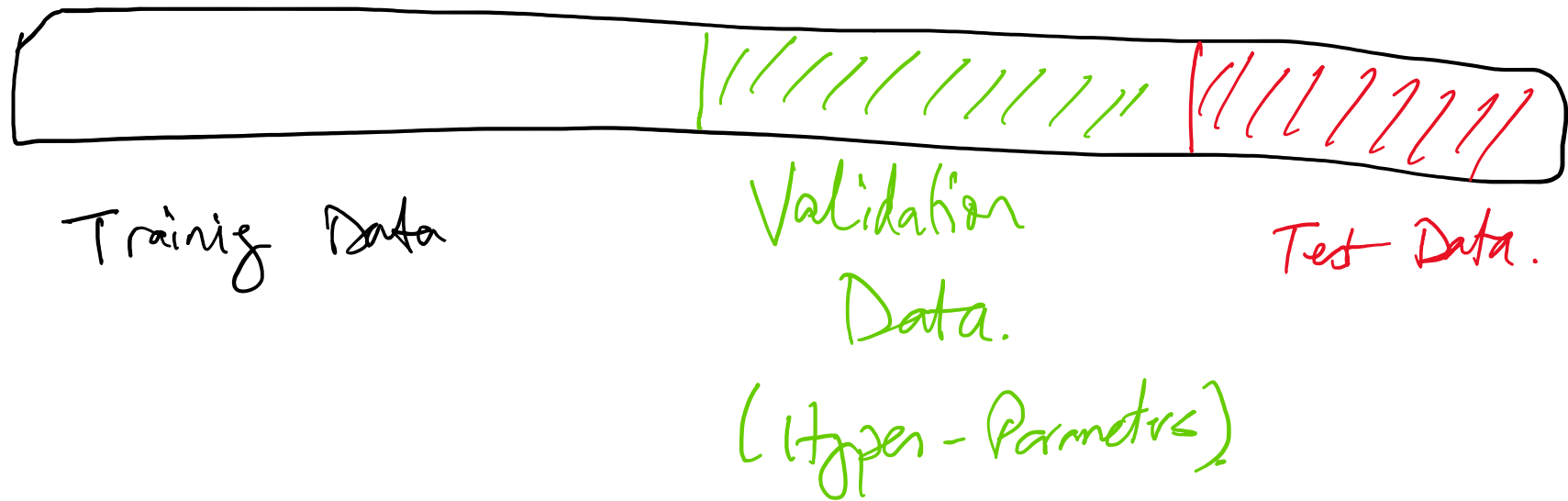
Randomly
Split



Split Samples
Based on Time



Splitting the Data for Meta-Learning



Overfitting can occur in meta-learning, too!

Solution: Cross-Validation

- **Goal:** Avoid over-fitting to the validation set during meta-learning
- **Approach:** Perform meta-learning using different subsets of the training data for the validation set.
- Two general approaches
 - **K-Fold** Cross Validation
 - **Leave-One Out** Cross-Validation: mostly useful for small datasets



Summary

- Machine learning models typically have **hyper-parameters** that can improve the performance of the models.
- Training to optimize both model parameters and hyper-parameters on the training data can result in a model that is overfit.
- We can make sure that a model is not overfit by holding out a validation set from the training set

- 5 – logistic regression – DNS query prediction
- 6 – Naïve Bayes – Spam filtering
- 6 – Naïve Bayes – Network traffic (DNS req/resp using simple NB, predict task using complementNB)
- 7 – Nearest-Neighbors-Trees -- Application to IoT privacy